

2025 年春 本科 Linux 期末大作业

作业要求

请大家认真阅读，按照要求完成：

1. 上机操作，作业以本 doc 文档为基础，写下源码，并在文档里提供源码运行后的拷屏结果。提交作业建议是[可提取文本内容的 pdf](#) 文件，便于老师拷贝作者的源码在机器上运行出结果。
2. 禁止相互抄袭，被抄袭的题目，无论抄袭者还是被抄袭者，一经发现，都被扣分。
3. 请于 **2025 年 6 月 20 日（周五 24:00）** 之前交作业。
4. 作业提交给 yuandong211@qq.com

为便于整理，email 提交方式：

邮件标题必须为 linux 大作业-姓名-班级-学号，以文件附件形式提交, 附件的文件名也要按照 linux 大作业-姓名-班级-学号 起名字。邮件设立了自动回复功能，如果收不到自动回复，请联系老师。作业只允许提交一次，不要提交多次。

操作系统环境：

AutoDL 租赁云计算设备

镜像	PyTorch 2.1.2 Python 3.10(ubuntu22.04) CUDA 11.8 更换
GPU	RTX 4090(24GB) * 1 升降配置
CPU	16 vCPU Intel(R) Xeon(R) Gold 6430
内存	120GB
硬盘	系统盘: 30 GB 数据盘: 免费:50GB 付费:0GB 扩容 缩容
附加磁盘	无
端口映射	无
自定义服务	
端口协议	http 修改
网络	同一地区实例共享带宽
计费方式	按量计费
费用	¥ 2.40/时

一、基础命令（30 分）

1. 登录你的机器，看一下你目前的位置，打印环境变量 PATH。创建 project 目录，进入 project 目录。在当前目录下创建 test.sh 文件并写入 'hello world!'，然后用输出重定向追加

写入 'Linux is a very practical course.'。将当前的 test.sh 文件复制到上一级目录中，再将当前目录下的 test.sh 重命名为 test_bak.sh。修改 test_bak.sh 权限：将读写执行权限分配给用户所有者，将读取执行权限分配给文件所属组和其他用户。

解：

查看当前位置：pwd

```
root@autodl-container-2db3448989-2d60d836:~# pwd
/root
```

打印环境变量 PATH：echo \$PATH

```
root@autodl-container-2db3448989-2d60d836:~# echo $PATH
/root/miniconda3/bin:/usr/local/bin:/usr/local/nvidia/bin:/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

创建 project 目录，查看创建情况并进入：mkdir project ;ls; cd project

```
root@autodl-container-2db3448989-2d60d836:~# mkdir project ;ls; cd project
autodl-pub  autodl-tmp  code  code.zip  miniconda3  project  temp_convert.sh  test.sh  tf-logs
root@autodl-container-2db3448989-2d60d836:~/project#
```

创建 test.sh 文件，写入内容并查看 test.sh 内容：echo 'hello world!' > test.sh; cat test.sh

```
root@autodl-container-2db3448989-2d60d836:~/project# echo 'hello world!' > test.sh; cat test.sh
hello world!
```

追加写入内容并查看 test.sh 内容：echo 'Linux is a very practical course.' >> test.sh; cat

test.sh

```
root@autodl-container-2db3448989-2d60d836:~/project# echo 'Linux is a very practical course.' >> test.sh; cat test.sh
hello world!
Linux is a very practical course.
```

复制文件到上一级目录，查看上一级目录内容：cp test.sh ../; ls ../

```
root@autodl-container-2db3448989-2d60d836:~/project# cp test.sh ../; ls ../
autodl-pub  autodl-tmp  code  code.zip  miniconda3  project  temp_convert.sh  test.sh  tf-logs
```

重命名当前目录下的 test.sh，查看当前目录情况：mv test.sh test_bak.sh; ls

```
root@autodl-container-2db3448989-2d60d836:~/project# mv test.sh test_bak.sh; ls
test_bak.sh
```

修改权限并查看权限情况：用户 rwx(7)，组 rx(5)，其他 rx(5)：chmod 755 test_bak.sh; ls

-l test_bak.sh

```
root@autodl-container-2db3448989-2d60d836:~/project# chmod 755 test_bak.sh; ls -l test_bak.sh
-rwxr-xr-x 1 root root 47 Jun  2 22:08 test_bak.sh
```

2. 在 /var/log 目录下有多个日志文件，其中包含了系统的运行日志。目录结构如下请完成以下任务：

目录结构：

```
/var/log
├── auth.log
├── system.log
├── error_log
├── application.log
└── ...

/home/user/logs
```

(1) 将所有以 .log 为后缀的日志文件合并到一个名为 system_logs.txt 的文件中。

解：

合并所有.log 文件到 system_logs.txt 并查看文件开头内容：

cat /var/log/*.log > system_logs.txt; tail -n 10 system_logs.txt

```

root@autodl-container-2db3448989-2d60d836:~# cat /var/log/*.log > system_logs.txt; tail -n 10 system_logs.txt
2025-05-30 11:02:09 status half-configured bc:amd64 1.07.1-3build1
2025-05-30 11:02:09 status installed bc:amd64 1.07.1-3build1
/usr/share/fonts: caching, new cache contents: 0 fonts, 1 dirs
/usr/share/fonts/truetype: caching, new cache contents: 0 fonts, 1 dirs
/usr/share/fonts/truetype/dejavu: caching, new cache contents: 6 fonts, 0 dirs
/usr/local/share/fonts: caching, new cache contents: 0 fonts, 0 dirs
/usr/share/fonts/truetype: skipping, looped directory detected
/usr/share/fonts/truetype/dejavu: skipping, looped directory detected
/var/cache/fontconfig: cleaning cache directory
fc-cache: succeeded

```

(2) 统计后缀为.log 的文件数量, 再寻找以 error 开头, 中间有 1 个字符, 2 个英文字母, 以 g 结尾的文件。

解:

统计.log 文件数量:

ls /var/log/*.log | wc -l

```

root@autodl-container-2db3448989-2d60d836:~# ls /var/log/*.log | wc -l
4

```

寻找符合模式的文件:

ls /var/log/error?[a-zA-Z][a-zA-Z]g

```

root@autodl-container-2db3448989-2d60d836:~# ls /var/log/error?[a-zA-Z][a-zA-Z]g
ls: cannot access '/var/log/error?[a-zA-Z][a-zA-Z]g': No such file or directory

```

(3) 创建 test_logs 目录, 并将找到的这些.log 文件打包在 test_logs 目录中的一个打包文件里

解:

创建 test_logs 目录并打包.log 文件:

mkdir test_logs

ls

tar -cf test_logs/log.tar /var/log/*.log

ls test_logs/

```

root@autodl-container-2db3448989-2d60d836:~# mkdir test_logs
root@autodl-container-2db3448989-2d60d836:~# ls
autodl-pub  autodl-tmp  code  code.zip  miniconda3  project  system_logs.txt  temp_convert.sh  test.sh  test_logs  tf-logs
root@autodl-container-2db3448989-2d60d836:~# tar -cf test_logs/log.tar /var/log/*.log
tar: Removing leading '/' from member names
tar: Removing leading '/' from hard link targets
root@autodl-container-2db3448989-2d60d836:~# ls test_logs/
log.tar

```

(4) 进入 test_logs 目录, 解压 log.tar, 查找并显示包含关键字 CRITICAL ERROR 的所有日志行, 然后返回上级目录, 强制删除 test_logs 目录。

解:

进入 test_logs 目录, 解压并查找关键字:

cd test_logs

tar -xf log.tar

grep -r "CRITICAL ERROR" . || echo "未找到包含'CRITICAL ERROR'的日志行"

```

root@autodl-container-2db3448989-2d60d836:~# cd test_logs
root@autodl-container-2db3448989-2d60d836:~/test_logs# tar -xf log.tar
root@autodl-container-2db3448989-2d60d836:~/test_logs# grep -r "CRITICAL ERROR" . || echo "未找到包含'CRITICAL ERROR'的日志行"
未找到包含'CRITICAL ERROR'的日志行

```

返回上级目录并强制删除 test_logs:

cd ..

ls

rm -rf test_logs

ls

```

root@autodl-container-2db3448989-2d60d836:~/test_logs# cd ..
root@autodl-container-2db3448989-2d60d836:~# ls
autodl-pub  autodl-tmp  code  code.zip  miniconda3  project  system_logs.txt  temp_convert.sh  test.sh  test_logs  tf-logs
root@autodl-container-2db3448989-2d60d836:~# rm -rf test_logs
root@autodl-container-2db3448989-2d60d836:~# ls
autodl-pub  autodl-tmp  code  code.zip  miniconda3  project  system_logs.txt  temp_convert.sh  test.sh  tf-logs

```

3.你作为系统管理员，收到开发团队反馈称服务器出现性能下降问题。请完成以下诊断任务：

（1）查看系统发行版本和系统架构。

解：

执行：

lsb_release -a

uname -m

```

root@autodl-container-2db3448989-2d60d836:~# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.1 LTS
Release:        22.04
Codename:       jammy
root@autodl-container-2db3448989-2d60d836:~# uname -m
x86_64

```

（2）以易读的方式查看所有磁盘分区的使用情况。

解：

执行：

df -h

```

root@autodl-container-2db3448989-2d60d836:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay          30G   2.2G   28G   8% /
tmpfs            64M    0    64M   0% /dev
shm             2.0G    0    2.0G   0% /dev/shm
/dev/md0        150M   4.0K  150M   1% /init
AutoFS:fs1       10T   4.4T   5.7T  44% /autodl-pub/data
tmpfs           504G   12K   504G   1% /proc/driver/nvidia
tmpfs           504G    0   504G   0% /etc/nvidia/nvidia-application-profiles-rc.d
/dev/sda2       438G   26G   390G   7% /usr/bin/nvidia-smi
tmpfs           504G    0   504G   0% /proc/asound
tmpfs           504G    0   504G   0% /proc/acpi
tmpfs           504G    0   504G   0% /proc/scsi
tmpfs           504G    0   504G   0% /sys/firmware

```

（3）查看 CPU 信息。

解：

执行：

lscpu

```

root@autodl-container-2db3448989-2d60d836:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          52 bits physical, 57 bits virtual
Byte Order:             Little Endian
CPU(s):                 128
On-line CPU(s) list:    0-127
Vendor ID:              GenuineIntel
Model name:             Intel(R) Xeon(R) Gold 6430
CPU family:             6
Model:                  143
Thread(s) per core:     2
Core(s) per socket:     32
Socket(s):              2
Stepping:               8
CPU max MHz:            3400.0000
CPU min MHz:            800.0000
BogoMIPS:               4200.00
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe sy
                        scall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf tsc
                        _known_freq pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid dca sse4_1 sse4_2 x2api
                        c movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l3 cat_l2 cdp_l3
                        invpcid_single intel_ppin cdp_l2 ssbd mba ibrs ibpb stibp ibrs_enhanced tpr_shadow vnmi flexpriority ept vpid ept_ad fsgs
                        base tsc_adjust bmi1 avx2 smep bmi2 erms invpcid cqm rdt_a avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb int
                        el_pt avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total cqm_mbm_local s
                        plit_lock_detect avx_vnni avx512_bf16 wbnoinvd dtherm ida arat pln pts hwp hwp_act_window hwp_epp hwp_pkg_req avx512vbmi u
                        mip plru ospke waitpkg avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg tme avx512_vpoptopq la57 rdpid bus_lock
                        _detect cldemote movdiri movdir64b enqcmd fsrm md_clear serialize tsxldtrk pconfig arch_lbr amx_bf16 avx512_fp16 amx_tile
                        amx_int8 flush_lld arch_capabilities

Virtualization features:
Virtualization:         VT-x
Caches (sum of all):
  L1d:                   3 MiB (64 instances)
  L1i:                   2 MiB (64 instances)
  L2:                    128 MiB (64 instances)
  L3:                    120 MiB (2 instances)
NUMA:
NUMA node(s):           2
NUMA node0 CPU(s):      0-31,64-95
NUMA node1 CPU(s):      32-63,96-127
Vulnerabilities:
Gather data sampling:    Not affected
Itlb multihit:          Not affected
L1tf:                   Not affected
Mds:                    Not affected
Meltdown:               Not affected
Mmio stale data:        Not affected
Retbleed:               Not affected
Spec rstack overflow:    Not affected
Spec store bypass:      Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Spectre v1:             Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:             Mitigation; Enhanced IBRS, IBPB conditional, RSB filling, PBSRB-eIBRS SW sequence
Srbds:                  Not affected

```

(4) 以易读的方式查看系统运行内存使用情况。

解:

执行:

free -h

```

root@autodl-container-2db3448989-2d60d836:~# free -h
               total        used        free      shared  buff/cache   available
Mem:           1.0Ti        48Gi        230Gi        6.9Gi        728Gi        945Gi
Swap:           0B           0B           0B

```

(5) 查看网络配置信息。

解:

执行:

ip addr show

```

root@autodl-container-2db3448989-2d60d836:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
59109: eth0@if59110: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever

```

二、字符串操作（20 分）

1.现有文件 3.txt 如下:

```

F115!16201!1174113017250745 10.86.96.41 211.140.16.1 200703180718
F125!16202!1174113327151715 10.86.96.42 211.140.16.2 200703180728
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738

```

F245!16204!1174113847250745 10.86.96.44 211.140.16.4 200703180748
F355!16205!1174115827252725 10.86.96.45 211.140.16.5 200703180758
In the heart of the night, when the world is asleep
In the heart of the day, where the shadows creep
In the silence, a whisper, a secret to keep
In the silence, a promise, a truth to believe
In the heart of the journey, we find what we seek
But sometimes the path leads us back where we began

(1) 用 cut 和 sed 通过管道的方式或者用 awk, 把红色部分提取出来解:

先使用 cat 命令创建 3.txt 文件并检查创建效果:

```
cat > 3.txt << 'EOF'
```

F115!16201!1174113017250745 10.86.96.41 211.140.16.1 200703180718
F125!16202!1174113327151715 10.86.96.42 211.140.16.2 200703180728
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F245!16204!1174113847250745 10.86.96.44 211.140.16.4 200703180748
F355!16205!1174115827252725 10.86.96.45 211.140.16.5 200703180758
In the heart of the night, when the world is asleep
In the heart of the day, where the shadows creep
In the silence, a whisper, a secret to keep
In the silence, a promise, a truth to believe
In the heart of the journey, we find what we seek
But sometimes the path leads us back where we began
EOF

```
cat 3.txt
```

```
root@autodl-container-2db3448989-2d60d836:~# cat > 3.txt << 'EOF'
> F115!16201!1174113017250745 10.86.96.41 211.140.16.1 200703180718
F125!16202!1174113327151715 10.86.96.42 211.140.16.2 200703180728
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F245!16204!1174113847250745 10.86.96.44 211.140.16.4 200703180748
F355!16205!1174115827252725 10.86.96.45 211.140.16.5 200703180758
In the heart of the night, when the world is asleep
In the heart of the day, where the shadows creep
In the silence, a whisper, a secret to keep
In the silence, a promise, a truth to believe
In the heart of the journey, we find what we seek
But sometimes the path leads us back where we began
EOF
root@autodl-container-2db3448989-2d60d836:~# cat 3.txt
F115!16201!1174113017250745 10.86.96.41 211.140.16.1 200703180718
F125!16202!1174113327151715 10.86.96.42 211.140.16.2 200703180728
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F235!16203!1174113737250745 10.86.96.43 211.140.16.3 200703180738
F245!16204!1174113847250745 10.86.96.44 211.140.16.4 200703180748
F355!16205!1174115827252725 10.86.96.45 211.140.16.5 200703180758
In the heart of the night, when the world is asleep
In the heart of the day, where the shadows creep
In the silence, a whisper, a secret to keep
In the silence, a promise, a truth to believe
In the heart of the journey, we find what we seek
But sometimes the path leads us back where we began
```

再提取第 7 行第 17-27 个字符：

```
sed -n '7p' 3.txt | cut -c17-27
```

```
root@autodl-container-2db3448989-2d60d836:~# sed -n '7p' 3.txt | cut -c17-27
15827252725
```

(2) 对 3.txt 文件中的第 2 至第 6 行按 IP 地址字段（即第三列）出现的次数进行降序排序，并将排序后的结果输出到屏幕上。

解：

对第 2-6 行按 IP 地址（第三列）出现次数降序排序：

```
sed -n '2,6p' 3.txt | awk '{print $3}' | sort | uniq -c | sort -nr
```

```
root@autodl-container-2db3448989-2d60d836:~# sed -n '2,6p' 3.txt | awk '{print $3}' | sort | uniq -c | sort -nr
 3 211.140.16.3
 1 211.140.16.4
 1 211.140.16.2
```

(3) 将 3.txt 文件中的第 8 行至第 13 行 按每行 2 个单词输出，并添加到 3.txt 末尾。

解：

将第 8-13 行按每行 2 个单词输出并添加到 3.txt 末尾，然后展示附加的新内容：

```
sed -n '8,13p' 3.txt | awk '{for(i=1;i<=NF;i+=2) print $i, $(i+1)}' >> 3.txt
```

```
tail -n 32 3.txt
```

```
we began
root@autodl-container-2db3448989-2d60d836:~# sed -n '8,13p' 3.txt | awk '{for(i=1;i<=NF;i+=2) print $i, $(i+1)}' >> 3.txt
root@autodl-container-2db3448989-2d60d836:~# tail -n 32 3.txt
In the
heart of
the night,
when the
world is
asleep
In the
heart of
the day,
where the
shadows creep
In the
silence, a
whisper, a
secret to
keep
In the
silence, a
promise, a
truth to
believe
In the
heart of
the journey,
we find
what we
seek
But sometimes
the path
leads us
back where
we began
```

2. 假设有字符串 `source_str="user#johndoe, age#30 job#developer`

`EMAIL#john@example.com,"`，请完成以下操作：

(1) 使用 shell 内置函数输出字符串变量 `$source_str` 的长度；

解：

定义字符串变量：

```
source_str="user#johndoe, age#30 job#developer EMAIL#john@example.com,"
```

使用 shell 内置函数输出字符串长度：

```
echo ${#source_str}
```

```
root@autodl-container-2db3448989-2d60d836:~# source_str="user#johndoe, age#30 job#developer EMAIL#john@example.com,"
root@autodl-container-2db3448989-2d60d836:~# echo ${#source_str}
58
```


(2) 使用 shell 内置函数从左侧来看, 把红色部分提取出来, 从右侧来看, 把红色部分提取出来:

从左侧提取红色部分:

```
echo ${source_str%% EMAIL*}
```

```
root@autodl-container-2db3448989-2d60d836:~# echo ${source_str%% EMAIL*}
user#johndoe, age#30 job#developer
```

从右侧提取红色部分:

```
echo ${source_str#*job#30 }
```

```
root@autodl-container-2db3448989-2d60d836:~# echo ${source_str#*30 }
job#developer EMAIL#john@example.com,
```

(3) 使用 shell 内置函数分别输出子串 job#developer 左边和右边的字符串;
解:

输出 job#developer 左边的字符串:

```
echo ${source_str%job#developer*}
```

```
root@autodl-container-2db3448989-2d60d836:~# echo ${source_str%job#developer*}
user#johndoe, age#30
```

输出 job#developer 右边的字符串:

```
echo ${source_str#*job#developer}
```

```
root@autodl-container-2db3448989-2d60d836:~# echo ${source_str#*job#developer}
EMAIL#john@example.com,
```

(4) 把字符串 source_str 里的所有 '#' 替换成 ':' , 删除多余的 ',' , 再将 'EMAIL' 改成小写, 最后将新串赋给 new_str 变量。

将所有 '#' 替换成 ':', 删除句末的 ',', 然后将 'EMAIL' 改成小写, 输出最终结果:

```
temp_str=${source_str//#/:}
```

```
temp_str=${temp_str%,}
```

```
new_str=${temp_str//EMAIL/email}
```

```
echo $new_str
```

```
root@autodl-container-2db3448989-2d60d836:~# temp_str=${source_str//#/:}
root@autodl-container-2db3448989-2d60d836:~# temp_str=${temp_str%,}
root@autodl-container-2db3448989-2d60d836:~# new_str=${temp_str//EMAIL/email}
```

```
root@autodl-container-2db3448989-2d60d836:~# echo $new_str
user:johndoe, age:30 job:developer email:john@example.com
```

三、Shell 脚本 (30 分)

1. 批量重命名为时间戳文件名

任务要求:

请编写一个 Shell 脚本 rename_with_timestamp.sh, 实现对指定目录中所有文件 (不含子目录中的文件) 进行重命名; 每个文件将被重命名为<原始文件名不含扩展名>_YYYY-MM-DD-HH-MM-SS.<原始扩展名>的格式; 时间戳格式为: 年月日时分秒 (%Y%m%d%H%M%S)

脚本要求:

1) 脚本接受 1 个参数: 目录路径;

- 2) 按文件字母序处理所有文件;
- 3) 如果文件已经包含时间戳 (例如 xxx_2025-01-01-15-30-22.txt), 则跳过;
- 4) 输出每次重命名操作的日志;
- 5) 时间戳使用当前系统时间;

脚本运行方式:

```
chmod +x rename_with_timestamp.sh
./rename_with_timestamp.sh ./test_dir
```

运行示例:

```
$ ./rename_with_timestamp.sh ./test_files
Renaming 'a.txt' → 'a_2025-05-19-20-35-50.txt'
Renaming 'report.md' → 'report_2025-05-19-20-35-50.md'
Skipping 'b_2024-12-31-23-59-59.txt': Already contains timestamp
Renaming completed!
```

解:

- (1) 使用 vim 写入 ./rename_with_timestamp.sh 内容, 并赋予执行权限:

```
vim ./rename_with_timestamp.sh
(键入 i 开启 INSERT 模式, 输入脚本内容, 键入 ESC, 键入:wq 保存并退出)
chmod +x rename_with_timestamp.sh
```

```
root@autodl-container-2db3448989-2d60d836:~# vim ./rename_with_timestamp.sh
root@autodl-container-2db3448989-2d60d836:~# chmod +x rename_with_timestamp.sh
```

```
rename_with_timestamp.sh
```

```
#!/bin/bash
```

```
# 检查参数数量
```

```
if [ $# -ne 1 ]; then
    echo "用法: $0 <目录路径>"
    exit 1
fi
```

```
# 获取目录路径
```

```
target_dir="$1"
```

```
# 检查目录是否存在
```

```
if [ ! -d "$target_dir" ]; then
    echo "错误: 目录 '$target_dir' 不存在"
    exit 1
fi
```

```
# 获取当前时间戳
```

```
timestamp=$(date +"%Y%m%d%H%M%S")
```

```
formatted_timestamp=$(date +"%Y-%m-%d-%H-%M-%S")

# 进入目标目录
cd "$target_dir" || exit 1

# 获取所有文件（按字母序排序，不包含子目录）
files=$(find . -maxdepth 1 -type f -exec basename {} \; | sort)

# 检查是否有文件需要处理
if [ ${#files[@]} -eq 0 ]; then
    echo "目录中没有找到文件"
    exit 0
fi

# 处理每个文件
for file in "${files[@]"; do
    # 检查文件是否已经包含时间戳格式 (xxx_YYYY-MM-DD-HH-MM-SS.ext)
    if [[ "$file" =~ .*_[0-9]{4}-[0-9]{2}-[0-9]{2}-[0-9]{2}-[0-9]{2}-[0-9]{2}\. ]]; then
        echo "Skipping '$file': Already contains timestamp"
        continue
    fi

    # 获取文件名和扩展名
    filename=$(basename "$file")
    extension=""
    basename_without_ext="$filename"

    # 提取扩展名（如果存在）
    if [[ "$filename" == *.* ]]; then
        extension=".${filename##*}"
        basename_without_ext="${filename%.*}"
    fi

    # 构造新文件名
    new_filename="${basename_without_ext}_${formatted_timestamp}${extension}"

    # 检查新文件名是否已存在
    if [ -e "$new_filename" ]; then
        echo "警告: 文件 '$new_filename' 已存在, 跳过 '$file'"
        continue
    fi

    # 重命名文件
    mv "$file" "$new_filename"
```

```
    echo "Renaming '$file' → '$new_filename'"
done

echo "Renaming completed!"
```

(2) 脚本测试:

创建实验目录和文件:

```
mkdir test_files
ls
cd test_files
touch a.txt report.md b_2025-06-02-18-00-00.txt
ls
cd ..
```

```
root@autodl-container-2db3448989-2d60d836:~# mkdir test_files
root@autodl-container-2db3448989-2d60d836:~# ls
3.txt      autodl-tmp  code.zip    project     system_logs.txt  test.sh      tf-logs
autodl-pub code        miniconda3 rename_with_timestamp.sh temp_convert.sh test_files
root@autodl-container-2db3448989-2d60d836:~# cd test_files
root@autodl-container-2db3448989-2d60d836:~/test_files# touch a.txt report.md b_2025-06-02-18-00-00.txt
root@autodl-container-2db3448989-2d60d836:~/test_files# ls
a.txt  b_2025-06-02-18-00-00.txt  report.md
root@autodl-container-2db3448989-2d60d836:~/test_files# cd ..
```

运行脚本并查看运行效果:

```
./rename_with_timestamp.sh ./test_files
cd test_files
ls
```

```
root@autodl-container-2db3448989-2d60d836:~# ./rename_with_timestamp.sh ./test_files
Renaming 'a.txt' → 'a_2025-06-02-23-23-22.txt'
Skipping 'b_2025-06-02-18-00-00.txt': Already contains timestamp
Renaming 'report.md' → 'report_2025-06-02-23-23-22.md'
Renaming completed!
root@autodl-container-2db3448989-2d60d836:~# cd test_files
root@autodl-container-2db3448989-2d60d836:~/test_files# ls
a_2025-06-02-23-23-22.txt  b_2025-06-02-18-00-00.txt  report_2025-06-02-23-23-22.md
```

2. 统计文本文件中单词出现频率并输出前 N 个高频词

任务要求:

编写一个 Shell 脚本 `FrequencyCount.sh`, 用于分析指定文本文件中的单词出现频率, 并输出出现频率最高的前 N 个单词。程序需满足以下功能:

- 忽略大小写 ("The" 与 "the" 视为同一个单词);
- 忽略标点符号 (如 `,.?!;:-` 等);
- 每行输出一个单词及其频率, 格式为: 单词 频率;
- 结果按频率从高到低排序;

脚本接受以下参数:

第一个参数: 文本文件的路径; 第二个参数: 需要输出的前 N 个高频词数量;

示例:

对于一个文本文件 `FrequencyCount.txt`, 内容为:

The quick brown fox jumps over the lazy dog. The dog was not amused!

执行:

```
chmod +x FrequencyCount.sh
./FrequencyCount.sh FrequencyCount.txt 3
输出：
the 3
dog 2
quick 1
```

解：

(1) 使用 vim 写入 FrequencyCount.sh 内容，并赋予执行权限：

```
vim FrequencyCount.sh
```

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
chmod +x FrequencyCount.sh
```

```
root@autodl-container-2db3448989-2d60d836:~# vim FrequencyCount.sh
root@autodl-container-2db3448989-2d60d836:~# chmod +x FrequencyCount.sh
```

FrequencyCount.sh:

```
#!/bin/bash

# 检查参数数量
if [ $# -ne 2 ]; then
    echo "用法: $0 <文件路径> <前 N 个词数量>"
    exit 1
fi

# 获取参数
file_path="$1"
n="$2"

# 检查文件是否存在
if [ ! -f "$file_path" ]; then
    echo "错误: 文件 '$file_path' 不存在"
    exit 1
fi

# 检查 N 是否为正整数
if ! [[ "$n" =~ ^[1-9][0-9]*$ ]]; then
    echo "错误: N 必须是正整数"
    exit 1
fi

# 处理文本文件并统计词频
cat "$file_path" | \
    # 转换为小写
    tr '[:upper:]' '[:lower:]' | \
```

```
# 去除标点符号，保留字母、数字和空格
tr -d '[:punct:]' | \
# 将空格、制表符、换行符替换为换行符，每个单词一行
tr -s '[:space:]' '\n' | \
# 去除空行
grep -v '^$' | \
# 排序
sort | \
# 统计每个单词出现的次数
uniq -c | \
# 按频率从高到低排序（数字排序，逆序）
sort -nr | \
# 获取前 N 个结果
head -n "$n" | \
# 格式化输出：单词 频率
awk '{print $2, $1}'
```

(2) 脚本测试：

创建测试文件并查看测试情况：

```
echo "The quick brown fox jumps over the lazy dog. The dog was not amused!" >
FrequencyCount.txt
cat FrequencyCount.txt
```

```
root@autodl-container-2db3448989-2d60d836:~# echo "The quick brown fox jumps over the lazy dog. The dog was not amused!" > FrequencyCount.txt
root@autodl-container-2db3448989-2d60d836:~# cat FrequencyCount.txt
The quick brown fox jumps over the lazy dog. The dog was not amused!
```

运行脚本并查看运行效果：

```
./FrequencyCount.sh FrequencyCount.txt 3
```

```
root@autodl-container-2db3448989-2d60d836:~# ./FrequencyCount.sh FrequencyCount.txt 3
the 3
dog 2
was 1
```

3.ISBN-10 编号合法性验证

ISBN-10（International Standard Book Number）是图书的国际标准书号，由 10 位字符组成，校验规则如下：

格式：

共 10 位字符（前 9 位是数字，第 10 位可以是数字或字母 X）。

X 表示数字 10。

计算校验和：

将每一位数字与其所在的位置相乘（从左到右，位置从 1 到 10）：

$$\text{checksum} = 1 \times d_1 + 2 \times d_2 + 3 \times d_3 + \dots + 10 \times d_{10}$$

其中， $d_1, d_2, \dots, d_9$ 为前 9 位数字

d_{10} 为第 10 位，若为 X，视作 10

判断是否有效：

如果 $\text{checksum} \% 11 == 0$ ，则该 ISBN 是有效的。

任务要求：请编写 Shell 脚本 `validate_isbn.sh`，用于验证用户输入的 ISBN-10 编号 是否合法。程序满足：

- 3.1.脚本接受一个包含 10 个字符（可含空格或破折号 -）的字符串作为参数；
- 3.2.脚本会去除输入中的空格和破折号；
- 3.3.若输入不满 10 位数字（最后一位允许为 X，表示 10），则输出：Invalid input length；
- 3.4.根据 ISBN-10 校验算法判断是否合法：
 - 每一位数字从左到右编号为 1~10；
 - 若最后一位为 X，视作数字 10；
 - 将每一位数字乘以其位置序号，然后求和；
 - 若总和能被 11 整除，则为合法 ISBN，输出：Validation Passed；
 - 否则输出：Validation Failed。

脚本接受以下参数：

一个字符串形式的 ISBN-10 编号。

示例：

```
$ ./validate_isbn.sh "0-306-40615-2"
Validation Passed
$ ./validate_isbn.sh "0 321 14653 0"
Validation Passed
$ ./validate_isbn.sh "123456789X"
Validation Passed
$ ./validate_isbn.sh "1234567890"
Validation Failed
$ ./validate_isbn.sh "12345"
Invalid input length
```

(1) 使用 vim 写入 `./validate_isbn.sh` 内容，并赋予执行权限：

```
vim validate_isbn.sh
```

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
chmod +x validate_isbn.sh
```

```
root@autodl-container-2db3448989-2d60d836:~# vim validate_isbn.sh
root@autodl-container-2db3448989-2d60d836:~# chmod +x validate_isbn.sh
```

```
validate_isbn.sh
```

```
#!/bin/bash
```

```
# 检查参数数量
```

```
if [ $# -ne 1 ]; then
```

```
    echo "用法: $0 \<ISBN-10 编号>"
```

```
    exit 1
```

```
fi
```

```
# 获取输入的 ISBN
```

```

input_isbn="$1"

# 去除空格和破折号
clean_isbn=$(echo "$input_isbn" | tr -d ' -')

# 检查长度是否为 10 位
if [ ${#clean_isbn} -ne 10 ]; then
    echo "Invalid input length"
    exit 1
fi

# 检查前 9 位是否都是数字
first_nine="${clean_isbn:0:9}"
if ! [[ "$first_nine" =~ ^[0-9]{9}$ ]]; then
    echo "Invalid input length"
    exit 1
fi

# 检查第 10 位是否是数字或 X
last_char="${clean_isbn:9:1}"
if ! [[ "$last_char" =~ ^[0-9X]$ ]]; then
    echo "Invalid input length"
    exit 1
fi

# 计算校验和
checksum=0

# 计算前 9 位的贡献
for i in {0..8}; do
    digit="${clean_isbn:$i:1}"
    position=$((i + 1))
    contribution=$((digit * position))
    checksum=$((checksum + contribution))
done

# 处理第 10 位（可能是 X）
if [ "$last_char" = "X" ]; then
    checksum=$((checksum + 10 * 10))
else
    checksum=$((checksum + last_char * 10))
fi

# 检查校验和是否能被 11 整除

```



```

if [ $((checksum % 11)) -eq 0 ]; then
    echo "Validation Passed"
else
    echo "Validation Failed"
fi

```

(2) 脚本测试:

运行脚本并查看运行效果:

```

./validate_isbn.sh "0-306-40615-2"
./validate_isbn.sh "0 321 14653 0"
./validate_isbn.sh "123456789X"
./validate_isbn.sh "1234567890"
./validate_isbn.sh "12345"

```

```

root@autodl-container-2db3448989-2d60d836:~# ./validate_isbn.sh "0-306-40615-2"
Validation Passed
root@autodl-container-2db3448989-2d60d836:~# ./validate_isbn.sh "0 321 14653 0"
Validation Passed
root@autodl-container-2db3448989-2d60d836:~# ./validate_isbn.sh "123456789X"
Validation Passed
root@autodl-container-2db3448989-2d60d836:~# ./validate_isbn.sh "1234567890"
Validation Failed
root@autodl-container-2db3448989-2d60d836:~# ./validate_isbn.sh "12345"
Invalid input length

```

四、Makefile 工程 (20)

1. 学生成绩管理系统

设计一个 C 语言项目，实现一个简单的学生成绩管理系统，实现如下功能模块:

1. 从指定文件中读取学生成绩 (0~100 的整数);
2. 实现计算: 学生成绩的平均值 (average.c), 及格学生的数量 (pass_count.c), 最低成绩的学生分数 (min_score.c)。

src/average.c: 计算平均值

src/pass_count.c: 统计 >=60 的人数

src/min_score.c: 找出最低成绩

include/average.h、include/pass_count.h、include/min_score.h 功能模块对应的头文件

src/main.c: 负责读取文件, 将数据传给上述模块处理, 并输出结果

Makefile: 支持自动化构建

结构要求:

所有 .c 源文件放在 src/ 目录中;

所有头文件 .h 放在 include/ 目录中;

创建一个 Makefile, 支持以下功能:

编译所有源文件并生成可执行程序 score_stat, 运行方式: ./score_stat

示例数据文件 scores.txt:

90

82

56
73
60
48
95
35
68
77
59

使用:

make

./score_app scores.txt

输出显示:

Total students: 11

Average score: 68.27

Pass count (≥ 60): 7

Minimum score: 35

(1) 设计并创建项目目录结构，本项目设计结构如下

student_score_system/	
— Makefile	定义 build 流程
— src/	目录: 包含所有 C 源文件
— main.c	主程序, 负责文件读取和结果输出
— average.c	计算学生成绩平均值的实现
— pass_count.c	统计及格学生数量 (≥ 60 分) 的实现
— min_score.c	查找最低成绩的实现
— include/	目录: 包含所有头文件
— average.h	计算学生成绩平均值的模块头文件
— pass_count.h	统计及格学生数量的模块头文件
— min_score.h	查找最低成绩的模块头文件
— scores.txt	示例数据文件

使用指令创建上述结构的项目:

```
mkdir student_score_system
cd student_score_system
mkdir src
touch src/main.c
touch src/average.c
touch src/pass_count.c
touch src/min_score.c
mkdir include
touch include/average.h
touch include/pass_count.h
touch include/min_score.h
touch Makefile
touch scores.txt
```

```

root@autodl-container-2db3448989-2d60d836:~# mkdir student_score_system
root@autodl-container-2db3448989-2d60d836:~# cd student_score_system
root@autodl-container-2db3448989-2d60d836:~/student_score_system# mkdir src
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch src/main.c
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch src/average.c
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch src/pass_count.c
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch src/min_score.c
root@autodl-container-2db3448989-2d60d836:~/student_score_system# mkdir include
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch include/average.h
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch include/pass_count.h
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch include/min_score.h
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch Makefile
root@autodl-container-2db3448989-2d60d836:~/student_score_system# touch scores.txt

```

使用指令查看项目创建情况：

tree

```

root@autodl-container-2db3448989-2d60d836:~/student_score_system# tree
.
├── Makefile
├── include
│   ├── average.h
│   ├── min_score.h
│   └── pass_count.h
├── scores.txt
└── src
    ├── average.c
    ├── main.c
    ├── min_score.c
    └── pass_count.c

2 directories, 9 files

```

(2) 写入 Makefile 文件：

vim Makefile

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```

root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim Makefile

```

Makefile:

```

# 编译器和编译选项
CC = gcc
CFLAGS = -Wall -Wextra -g -I$(INC_DIR)

# 目录定义
SRC_DIR = src
INC_DIR = include
OBJ_DIR = obj
BIN_DIR = .

# 目标可执行文件
TARGET = $(BIN_DIR)/score_stat

# 源文件和目标文件
SRCS = $(wildcard $(SRC_DIR)/*.c)
OBJS = $(patsubst $(SRC_DIR)/%.c, $(OBJ_DIR)/%.o, $(SRCS))

```

```
# 默认目标
all: $(OBJ_DIR) $(TARGET)

# 创建目标文件目录
$(OBJ_DIR):
    @mkdir -p $(OBJ_DIR)

# 链接生成可执行文件
$(TARGET): $(OBJS)
    @echo "Linking $@"
    @$(CC) $(CFLAGS) -o $@ $^
    @echo "Build complete: $@"

# 编译规则: .c -> .o
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    @echo "Compiling $<..."
    @$(CC) $(CFLAGS) -c $< -o $@
```

(3) 在 include/ 目录中写入.h 头文件:

创建 include/average.h:

vim include/average.h

(键入 i 开启 INSERT 模式, 输入脚本内容, 键入 ESC, 键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim include/average.h
```

include/average.h:

```
/**
 * @file average.h
 * @brief 计算学生成绩平均值的模块头文件
 */

#ifndef AVERAGE_H
#define AVERAGE_H

/**
 * @brief 计算学生成绩的平均值
 * @param scores 成绩数组
 * @param count 学生人数
 * @return 平均成绩 (double 类型)
 */
double calculate_average(const int *scores, int count);

#endif /* AVERAGE_H */
```

创建 include/pass_count.h:

vim include/pass_count.h

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim include/pass_count.h
```

include/pass_count.h:

```
/**
 * @file pass_count.h
 * @brief 统计及格学生数量的模块头文件
 */

#ifndef PASS_COUNT_H
#define PASS_COUNT_H

/**
 * @brief 统计及格学生的数量（成绩 >= 60）
 * @param scores 成绩数组
 * @param count 学生人数
 * @return 及格学生数量
 */
int count_pass_students(const int *scores, int count);

#endif /* PASS_COUNT_H */
```

创建 include/min_score.h:

vim include/min_score.h

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim include/min_score.h
```

include/min_score.h:

```
/**
 * @file min_score.h
 * @brief 查找最低成绩的模块头文件
 */

#ifndef MIN_SCORE_H
#define MIN_SCORE_H

/**
 * @brief 查找学生中的最低成绩
 * @param scores 成绩数组
 * @param count 学生人数
 * @return 最低成绩
 */
int find_min_score(const int *scores, int count);

#endif /* MIN_SCORE_H */
```

(4) 在 src/ 目录中写入.c 源文件:

vim src/main.c

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim src/main.c
```

src/main.c:

```
/**
 * @file main.c
 * @brief 学生成绩管理系统主程序
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include "average.h"
#include "pass_count.h"
#include "min_score.h"

#define MAX_STUDENTS 1000
#define MAX_LINE_LENGTH 10

/**
 * @brief 从文件读取学生成绩
 * @param filename 文件名
 * @param scores 存储成绩的数组
 * @param max_count 数组最大容量
 * @return 实际读取的成绩数量，出错返回-1
 */
static int read_scores_from_file(const char *filename, int *scores, int max_count) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        fprintf(stderr, "Error: Cannot open file '%s': %s\n",
                filename, strerror(errno));
        return -1;
    }

    int count = 0;
    char line[MAX_LINE_LENGTH];

    while (fgets(line, sizeof(line), fp) != NULL && count < max_count) {
        // 去除换行符
        line[strcspn(line, "\n")] = '\0';

        // 跳过空行
        if (strlen(line) == 0) {
```

```

        continue;
    }

    // 转换为整数
    char *endptr;
    long score = strtol(line, &endptr, 10);

    // 检查转换是否成功
    if (*endptr != '\0') {
        fprintf(stderr, "Warning: Invalid score format at line %d: '%s'\n",
            count + 1, line);
        continue;
    }

    // 检查成绩范围
    if (score < 0 || score > 100) {
        fprintf(stderr, "Warning: Score out of range at line %d: %ld\n",
            count + 1, score);
        continue;
    }

    scores[count++] = (int)score;
}

fclose(fp);
return count;
}

/**
 * @brief 主函数
 */
int main(int argc, char *argv[]) {
    // 检查命令行参数
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <score_file>\n", argv[0]);
        return EXIT_FAILURE;
    }

    // 分配成绩数组
    int scores[MAX_STUDENTS];

    // 读取成绩数据
    int student_count = read_scores_from_file(argv[1], scores, MAX_STUDENTS);
    if (student_count == -1) {

```

```

        return EXIT_FAILURE;
    }

    if (student_count == 0) {
        fprintf(stderr, "Error: No valid scores found in file\n");
        return EXIT_FAILURE;
    }

    // 计算统计数据
    double avg = calculate_average(scores, student_count);
    int pass_count = count_pass_students(scores, student_count);
    int min_score = find_min_score(scores, student_count);

    // 输出结果
    printf("Total students: %d\n", student_count);
    printf("Average score: %.2f\n", avg);
    printf("Pass count (>=60): %d\n", pass_count);
    printf("Minimum score: %d\n", min_score);

    return EXIT_SUCCESS;
}

```

vim src/average.c

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim src/average.c
```

src/average.c:

```

/**
 * @file average.c
 * @brief 计算学生成绩平均值的实现
 */

#include "average.h"
#include <stddef.h>

double calculate_average(const int *scores, int count) {
    if (scores == NULL || count <= 0) {
        return 0.0;
    }

    long long sum = 0; // 使用 long long 避免溢出

    for (int i = 0; i < count; i++) {
        sum += scores[i];
    }
}

```



```
    return (double)sum / count;
}
```

vim src/pass_count.c

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim src/pass_count.c
```

src/pass_count.c:

```
/**
 * @file pass_count.c
 * @brief 统计及格学生数量的实现
 */

#include "pass_count.h"
#include <stddef.h>

#define PASS_THRESHOLD 60

int count_pass_students(const int *scores, int count) {
    if (scores == NULL || count <= 0) {
        return 0;
    }

    int pass_count = 0;

    for (int i = 0; i < count; i++) {
        if (scores[i] >= PASS_THRESHOLD) {
            pass_count++;
        }
    }

    return pass_count;
}
```

vim src/min_score.c

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim src/min_score.c
```

src/min_score.c:

```
/**
 * @file min_score.c
 * @brief 查找最低成绩的实现
 */

#include "min_score.h"
```

```
#include <limits.h>
#include <stddef.h>

int find_min_score(const int *scores, int count) {
    if (scores == NULL || count <= 0) {
        return -1; // 错误返回值
    }

    int min_score = INT_MAX;

    for (int i = 0; i < count; i++) {
        if (scores[i] < min_score) {
            min_score = scores[i];
        }
    }

    return min_score;
}
```

(5) 写入示例数据文件:

vim scores.txt

(键入 i 开启 INSERT 模式, 输入脚本内容, 键入 ESC, 键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/student_score_system# vim scores.txt
```

scores.txt:

```
90
82
56
73
60
48
95
35
68
77
59
```

(6) 编译测试脚本效果:

make

./score_app scores.txt

```

root@autodl-container-2db3448989-2d60d836:~/student_score_system# make
Compiling src/average.c...
Compiling src/main.c...
Compiling src/min_score.c...
Compiling src/pass_count.c...
Linking score_app...
Build complete: score_app
root@autodl-container-2db3448989-2d60d836:~/student_score_system# ./score_app scores.txt
Total students: 11
Average score: 67.55
Pass count (>=60): 7
Minimum score: 35

```

2. 实现两个向量的点积（Dot Product）计算

向量点积公式：

设有两个长度为 n 的向量： $a = (a_1, a_2, \dots, a_n)$ ， $b = (b_1, b_2, \dots, b_n)$ ， 它们的点积为：

$$a \cdot b = \sum_{i=1}^n a_i \times b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

结果是一个标量，代表两个向量的乘积和。

题目要求：

a.dot_product.c 实现点积计算函数： `double dot_product(const double* vec1, const double* vec2, int length);`

b.main.c:

- 从命令行读取两个向量（先输入向量长度，然后输入向量元素，元素以空格分隔）
- 验证两个向量长度相同
- 调用点积函数计算结果
- 输出点积结果

c.输入长度必须是正整数，若长度不匹配或输入格式错误，程序应给出错误提示并退出

d.编写 Makefile 完成编译，生成可执行程序 `vector_dot`

项目结构：

源文件： `main.c`、 `dot_product.c`

头文件： `dot_product.h`

Makefile 文件

可执行程序名为： `vector_dot`

使用：

```

make          # 编译程序
./vector_dot

```

输入实例：

```

Enter the length of the vectors: 3
Enter elements of vector 1 separated by space: 1 2 3
Enter elements of vector 2 separated by space: 4 5 6
Dot product result: 32.0000

```

(1) 创建项目文件：

创建 Makefile 编译脚本，包含编译规则和选项：

vim Makefile

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/vector_dot# vim Makefile
```

Makefile:

```
# 编译器和编译选项
CC = gcc
CFLAGS = -Wall -Wextra -std=c99 -O2

# 目标程序名
TARGET = vector_dot

# 源文件和目标文件
SRCS = main.c dot_product.c
OBJS = $(SRCS:.c=.o)

# 默认目标
all: $(TARGET)

# 链接目标程序
$(TARGET): $(OBJS)
$(CC) $(OBJS) -o $(TARGET)

# 编译规则
%.o: %.c
$(CC) $(CFLAGS) -c $< -o $@

# 依赖关系
main.o: main.c dot_product.h
dot_product.o: dot_product.c dot_product.h
```

创建 dot_product.h 头文件，用于声明点积计算函数：

vim dot_product.h

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/vector_dot# vim dot_product.h
```

dot_product.h

```
#ifndef DOT_PRODUCT_H
#define DOT_PRODUCT_H

/**
 * 计算两个向量的点积
 * @param vec1 第一个向量
 * @param vec2 第二个向量
 * @param length 向量长度
```

```
* @return 点积结果
*/
double dot_product(const double* vec1, const double* vec2, int length);
#endif /* DOT_PRODUCT_H */
```

创建 dot_product.c 文件，用于实现点积计算函数：

vim dot_product.c

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/vector_dot# vim dot_product.c
```

dot_product.c:

```
#include "dot_product.h"

double dot_product(const double* vec1, const double* vec2, int length) {
    double result = 0.0;

    for (int i = 0; i < length; i++) {
        result += vec1[i] * vec2[i];
    }

    return result;
}
```

创建 main.c 文件，用于从命令行读取向量长度，读取两个向量的元素，验证输入合法性，调用点积函数并输出结果：

vim main.c

(键入 i 开启 INSERT 模式，输入脚本内容，键入 ESC，键入:wq 保存并退出)

```
root@autodl-container-2db3448989-2d60d836:~/vector_dot# vim main.c
```

main.c:

```
#include <stdio.h>
#include <stdlib.h>
#include "dot_product.h"

int main() {
    int length;
    double *vec1, *vec2;

    // 读取向量长度
    printf("Enter the length of the vectors: ");
    if (scanf("%d", &length) != 1 || length <= 0) {
        fprintf(stderr, "Error: Invalid length. Please enter a positive integer.\n");
        return 1;
    }

    // 分配内存
```

```

vec1 = (double*)malloc(length * sizeof(double));
vec2 = (double*)malloc(length * sizeof(double));

if (!vec1 || !vec2) {
    fprintf(stderr, "Error: Memory allocation failed.\n");
    free(vec1);
    free(vec2);
    return 1;
}

// 读取第一个向量
printf("Enter elements of vector 1 separated by space: ");
for (int i = 0; i < length; i++) {
    if (scanf("%lf", &vec1[i]) != 1) {
        fprintf(stderr, "Error: Invalid input format for vector 1.\n");
        free(vec1);
        free(vec2);
        return 1;
    }
}

// 读取第二个向量
printf("Enter elements of vector 2 separated by space: ");
for (int i = 0; i < length; i++) {
    if (scanf("%lf", &vec2[i]) != 1) {
        fprintf(stderr, "Error: Invalid input format for vector 2.\n");
        free(vec1);
        free(vec2);
        return 1;
    }
}

// 计算点积
double result = dot_product(vec1, vec2, length);

// 输出结果
printf("Dot product result: %.4f\n", result);

// 释放内存
free(vec1);
free(vec2);

return 0;
}

```

(2) 编译并测试程序效果:

make

./vector_dot

```
root@autodl-container-2db3448989-2d60d836:~/vector_dot# make
gcc -Wall -Wextra -std=c99 -O2 -c main.c -o main.o
gcc -Wall -Wextra -std=c99 -O2 -c dot_product.c -o dot_product.o
gcc main.o dot_product.o -o vector_dot
root@autodl-container-2db3448989-2d60d836:~/vector_dot# ./vector_dot
Enter the length of the vectors: 3
Enter elements of vector 1 separated by space: 1 2 3
Enter elements of vector 2 separated by space: 4 5 6
Dot product result: 32.0000
```